

Programación de Redes Neuronales Profundas - Guía Práctica

Objetivo del Caso Práctico

Desarrollar un modelo predictivo con redes neuronales profundas para detectar el riesgo de que un pedido se entregue fuera del plazo máximo, utilizando Python, TensorFlow y Keras.

1. Arquitectura del Modelo: Capas y Parámetros

1.1 Modelo Sequential en Keras

- **Clase Sequential:** Permite crear modelos capa por capa de forma lineal
- **Ventaja:** Sencillez para modelos con una única entrada y salida
- **Ejemplo básico:**

```
from keras.models import Sequential
model = Sequential()
```

1.2 Capa Dense (Totalmente Conectada)

- **Definición:** Cada neurona conectada con todas las neuronas de la capa anterior
- **Operación matemática:** `output = activation(dot(input, kernel) + bias)`
- **Parámetros principales:**
 - `units` : Número de neuronas (entero positivo)

- `activation` : Función de activación (ReLU, tanh, sigmoid, etc.)
- `input_shape` : Forma de los datos de entrada (solo en primera capa)
- `use_bias` : Booleano para incluir término de sesgo

1.3 Configuración de Neuronas por Capa

- **Capa de salida:**
 - Clasificación binaria: 1 neurona con activación sigmoid
 - Clasificación múltiple: N neuronas (una por clase) con activación softmax
- **Capas ocultas:** No hay regla fija, se recomienda experimentar
 - Comenzar con configuraciones sencillas
 - Aumentar complejidad gradualmente según necesidad
 - Orden de magnitud similar al número de variables de entrada

2. Entrenamiento del Modelo

2.1 Compilación del Modelo

- **Función de pérdida (Loss):**
 - Clasificación binaria: `binary_crossentropy`
 - Clasificación múltiple: `categorical_crossentropy`
- **Optimizadores:** Algoritmos que ajustan pesos durante el entrenamiento
 - Adam: Recomendado por defecto (combina ventajas de otros optimizadores)
 - SGD: Descenso de gradiente estocástico
 - RMSprop: Especialmente bueno para RNN
- **Métricas:** `accuracy` para problemas de clasificación

Ejemplo de compilación:

```
model.compile(optimizer='Adam',  
              loss='binary_crossentropy',  
              metrics=['accuracy'])
```

2.2 Parámetros de Entrenamiento

- **epochs:** Número de iteraciones completas sobre el dataset
- **batch_size:** Número de muestras por actualización de pesos
- Menor batch_size → más actualizaciones, más ruido
- Mayor batch_size → menos actualizaciones, más estable
- **shuffle:** Mezclar datos en cada epoch (evita sesgos)

3. Flujo de Trabajo Completo

3.1 Preparación de Datos

1. **Carga y limpieza:** Eliminar columnas irrelevantes (como IDs)
2. **Codificación de variables categóricas:** - Variables binarias: `replace()` para convertir a 0/1 - Variables múltiples: One-hot encoding con `pd.get_dummies()`
3. **Normalización:** Escalar datos con `StandardScaler()` para mejorar convergencia
4. **División:** Separar en conjuntos de entrenamiento (70%) y prueba (30%)

3.2 Construcción del Modelo - Ejemplo Práctico

```
model = Sequential()  
# Capa de entrada: 18 variables → 128 neuronas  
model.add(Dense(128, activation='tanh', input_shape=(18,)))  
# Capas ocultas  
model.add(Dense(320, activation='tanh'))  
model.add(Dense(512, activation='tanh'))  
model.add(Dense(320, activation='tanh'))  
model.add(Dense(128, activation='tanh'))  
# Capa de salida: 1 neurona para clasificación binaria  
model.add(Dense(1, activation='sigmoid'))
```

3.3 Resultados Obtenidos

- **Precisión final:** 68.39% en datos de prueba
- **Pérdida (loss):** 0.5044

- **Observación:** Resultado similar a modelos clásicos, pero las DNN brillan con:
- Grandes volúmenes de datos
- Datos no estructurados (imágenes, texto)
- Relaciones complejas entre variables

4. Recomendaciones Prácticas

4.1 Enfoque Iterativo

1. **Comenzar simple:** Modelo básico con pocas capas y neuronas
2. **Monitorizar:** Observar métricas durante entrenamiento
3. **Ajustar gradualmente:** Modificar un parámetro a la vez
4. **Documentar cambios:** Registrar qué ajustes mejoran/empeoran resultados

4.2 Funciones de Activación Comunes

- **ReLU:** Buena por defecto, evita problema de gradiente vanishing
- **tanh:** Rango $[-1, 1]$, útil para datos centrados en cero
- **sigmoid:** Para capa de salida en clasificación binaria (rango $[0, 1]$)

4.3 Consideraciones Importantes

- **Overfitting:** Si accuracy en entrenamiento $\gg$ accuracy en prueba
- **Underfitting:** Si ambas accuracy son bajas
- **Tiempo de cómputo:** Aumenta con número de capas/neuronas

Conclusión

Las redes neuronales profundas ofrecen gran flexibilidad para modelar relaciones complejas, pero requieren experimentación para encontrar la arquitectura óptima. El caso práctico muestra cómo implementar una DNN completa, desde la preparación de datos hasta la evaluación, destacando la importancia de la normalización, codificación adecuada y selección cuidadosa de hiperparámetros.

Resumen: Configuración y Entrenamiento de Redes Neuronales Profundas con Keras

Este documento proporciona una guía práctica para la construcción, configuración y entrenamiento de modelos de redes neuronales profundas utilizando la biblioteca Keras. Se centra en la toma de decisiones clave sobre hiperparámetros y componentes arquitectónicos.

1. Arquitectura de la Red y Funciones de Activación

Capa Flatten

- **Propósito:** Transforma datos de entrada multidimensionales (como imágenes) en un vector unidimensional, requisito para las capas densas (`Dense`).
- **Ejemplo:** Para un dataset de 1000 imágenes de 12x12 píxeles, la forma de entrada es `(1000, 12, 12)`. Tras `Flatten`, la salida es `(1000, 144)`.
- **Código:** `python model.add(keras.layers.Flatten(input_shape=(12, 12)))`

Funciones de Activación

La elección depende de la capa y el tipo de problema. Se recomienda una configuración base:

- * **Capas Internas:** Usar **ReLU** de forma predeterminada.
- * **Capa de Salida:**
 - * **Clasificación Binaria:** Usar **Sigmoid**.
 - * **Clasificación Multiclase:** Usar **Softmax**.

Ejemplo de modelo completo (clasificación de 4 frutas):

```
model = keras.Sequential()
model.add(keras.layers.Flatten(input_shape=(28, 28))) # Imágenes de 28x28
model.add(keras.layers.Dense(128, activation='relu')) # Capa oculta
model.add(keras.layers.Dense(4, activation='softmax')) # 4 neuronas p
```

2. Función de Coste o Pérdida (Loss)

La función de pérdida mide el error del modelo y guía el ajuste de los pesos durante el entrenamiento, buscando su minimización. En redes neuronales, es común usar variantes de la **Entropía Cruzada**.

Funciones Principales para Clasificación:

- **Binary Crossentropy** : Para problemas de **clasificación binaria**. Se combina con la función de activación **sigmoid** en la salida.
- **Categorical Crossentropy** : Para **clasificación múltiple** cuando las etiquetas están en formato **one-hot encoding**.
- **Sparse Categorical Crossentropy** : Para **clasificación múltiple** cuando las etiquetas son **enteros** (0, 1, 2...).

Para Problemas de Regresión (alternativas):

- **MeanSquaredError** : Penaliza fuertemente errores grandes.
- **MeanAbsolutePercentageError** : Error basado en porcentaje, fácil de interpretar.
- **MeanSquaredLogarithmicError** : Robusta ante valores atípicos (outliers).

3. Optimizadores y Tasa de Aprendizaje

El optimizador es el algoritmo que ajusta los pesos de la red para minimizar la función de coste, utilizando el **descenso del gradiente**.

Optimizadores Recomendados:

- **Adam**: Generalmente ofrece un buen comportamiento general y es una excelente opción inicial.
- **RMSprop**: Suele tener una convergencia inicial más rápida.

Tasa de Aprendizaje (Learning Rate)

Controla la magnitud de los ajustes en los pesos. Es un hiperparámetro crítico: * **Demasiado baja**: El entrenamiento es muy lento. * **Demasiado alta**: El

entrenamiento puede volverse inestable y no converger. * **Rango inicial recomendado:** Entre 0.0001 y 0.001 .

Ejemplo de configuración:

```
opt = keras.optimizers.Adam(learning_rate=0.001)
model.compile(loss='categorical_crossentropy', optimizer=opt)
```

4. Entrenamiento del Modelo

El entrenamiento se realiza con el método `.fit()` .

Parámetros Clave:

- **epochs** : Número de iteraciones completas sobre el conjunto de datos de entrenamiento. En cada época, el modelo busca un mínimo más profundo de la función de coste.
- **batch_size** : Número de muestras que se procesan antes de actualizar los pesos internos. Si no se especifica, el valor por defecto es 32.
 - **Cálculo:** Con 60,000 datos y `batch_size=40` , se tendrán 1,500 actualizaciones de pesos por época.

Ejemplo de llamada a `.fit()` :

```
model.fit(X_train, y_train, epochs=20, batch_size=40)
```

Flujo de Trabajo Recomendado:

1. **Empezar con modelos simples:** Comenzar con arquitecturas básicas (pocas capas y neuronas).
2. **Aprender de modelos existentes:** Basarse en configuraciones probadas por otros.
3. **Experimentar de forma controlada:** Ir modificando hiperparámetros (capas, neuronas, funciones de activación, optimizadores) de manera incremental y evaluando el impacto.

4. **Práctica:** La mejor forma de aprender es programando modelos, como el ejemplo de clasificación de moda (`Fashion MNIST`), y tratando de mejorar su precisión mediante ajustes.

Resumen: Entrenamiento de una Red Neuronal para Clasificación de Imágenes (Fashion MNIST)

Este documento detalla el proceso completo para construir, entrenar y evaluar una red neuronal profunda (DNN) utilizando TensorFlow/Keras para clasificar imágenes del conjunto de datos **Fashion MNIST**.

1. Carga y Exploración de los Datos

- **Conjunto de datos:** Fashion MNIST, que contiene 70,000 imágenes en escala de grises (60,000 para entrenamiento, 10,000 para prueba) de 10 categorías de ropa y accesorios.
- **Estructura de los datos:** Cada imagen tiene una resolución de 28x28 píxeles. Las etiquetas son números del 0 al 9 que corresponden a clases específicas.
- **Preprocesamiento:** Los valores de los píxeles (originalmente entre 0 y 255) se normalizan dividiendo por 255.0, escalándolos a un rango de [0, 1] para un entrenamiento más estable y rápido.
- **Visualización:** Se muestran ejemplos de las imágenes con sus etiquetas correspondientes (ej: 'T-shirt/top', 'Trouser', 'Sneaker') para comprender el conjunto de datos.

2. Construcción del Modelo de Red Neuronal

Se crea un modelo secuencial (`Sequential`) con las siguientes capas: 1. **Capa Flatten :** Transforma la imagen 2D (28, 28) en un vector 1D de 784 píxeles. Es la capa de entrada. 2. **Capa Dense (128 neuronas, `activation='relu'`):** Capa oculta totalmente conectada que utiliza la función de activación ReLU para introducir no linealidad y aprender patrones complejos. 3. **Capa Dense (10 neuronas,**

activation='softmax'): Capa de salida. La función `softmax` convierte las salidas en probabilidades (que suman 1) para cada una de las 10 clases, indicando la confianza del modelo en cada predicción.

3. Configuración y Entrenamiento del Modelo

- **Compilación (`compile`):** Se configura el proceso de aprendizaje.
 - **Optimizador:** `'Adam'` (algoritmo eficiente para ajustar los pesos de la red).
 - **Función de Pérdida (Loss):** `'sparse_categorical_crossentropy'` (adecuada para clasificación con etiquetas enteras).
 - **Métrica:** `'accuracy'` (precisión) para monitorear el porcentaje de clasificaciones correctas.
- **Entrenamiento (`fit`):** El modelo "aprende" ajustando sus pesos internos.
 - **Datos:** `train_images` y `train_labels`.
 - **Épocas (`epochs=5`):** Número de veces que el modelo revisa todo el conjunto de entrenamiento. En 5 épocas, la precisión en entrenamiento (`accuracy`) subió del **82.2% al 89.1%**, mientras la pérdida (`loss`) disminuyó.

4. Evaluación y Pruebas del Modelo

- **Evaluación (`evaluate`):** Se prueba el modelo con datos nunca vistos (`test_images` , `test_labels`). La **precisión en prueba (`test_acc`)** fue del **87.2%**, ligeramente inferior a la de entrenamiento, lo que indica un pequeño sobreajuste.
- **Predicciones (`predict`):** Se generan probabilidades para nuevas imágenes.
 - **Ejemplo:** Para la imagen de prueba en el índice 5, el modelo asignó una probabilidad del ~99.99% a la clase 1 (`'Trouser'`), que coincidía con la etiqueta real.
- **Visualización de Resultados:** Se crea una cuadrícula para visualizar 25 predicciones de prueba. Las etiquetas predichas se muestran en **verde si son correctas** y en **rojo si son incorrectas**, junto con la etiqueta verdadera entre paréntesis.

5. Conceptos Clave y Autoevaluación

- **Época (epoch):** Es el parámetro en `fit()` que define el número de iteraciones completas sobre todo el conjunto de datos de entrenamiento. Es la respuesta correcta a la pregunta de autoevaluación.
- **Loss (Pérdida):** Es la función objetivo (como `sparse_categorical_crossentropy`) que el optimizador intenta minimizar, no el número de iteraciones.

Propósito General: Esta guía proporciona un flujo de trabajo fundamental en Aprendizaje Automático: cargar datos, preprocesarlos, construir una arquitectura de red neuronal, configurar el aprendizaje, entrenar el modelo y finalmente evaluar su rendimiento en datos nuevos, utilizando un problema clásico de clasificación de imágenes como ejemplo práctico.